

Final_exp

December 12, 2025

1 Klasifikasi Penyakit pada Daun Padi dengan Momen warna, LBP, GLCM, PCA dan SVM

```
[2]: import os
import cv2
import time
import copy
import random
import pickle
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from skimage import feature
from sklearn.svm import SVC
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score, confusion_matrix, \
    classification_report, ConfusionMatrixDisplay
from sklearn.model_selection import StratifiedGroupKFold, cross_validate, \
    train_test_split, GridSearchCV
from scipy.stats import skew, kurtosis
```

Ini merupakan beberapa persiapan yang nantinya akan digunakan pada script berikutnya terutama dalam mengatur ukuran dan dataset mana yang nantinya akan dimuat serta penentuan nilai `seed` agar proses randomize dapat dihasilkan kembali dengan hasil yang sama.

```
[67]: IMG_SIZE = (222, 222)
random.seed(42)
np.random.seed(42)
main_dataset_path = 'datasets/local_dataset_no_duplicate'
is_trainSplitted = False
LABELS = np.array(os.listdir(f'{main_dataset_path}/train_images')) if \
    is_trainSplitted else np.array(os.listdir(main_dataset_path))
MAIN_DATASET = [main_dataset_path, 1400]
```

```

DATASETS = [['datasets/paddy-disease-classification-healthy', 1400], ['datasets/
↳rice_leaves_disease', 305]]
AUGMENTATION_TARGET = 5000
N_SPLITS = 5
EXPERIMENT_NAME = '251209 1547 Mixed On Field No Duplicate Normal No Indo'
TEST_IMAGES_PATH = f'datasets/result/{EXPERIMENT_NAME}/test_images'
BLIND_TEST_IMAGES_PATH = f'datasets/result/{EXPERIMENT_NAME}/test_images2'
AUGMENTED_IMAGES_PATH = f'datasets/result/{EXPERIMENT_NAME}/augmented_images'

RESULT_DICT_PATH = f'Results/{EXPERIMENT_NAME}'
Y_TRAIN_PATH = os.path.join(RESULT_DICT_PATH, 'y_train.csv')
Y_TEST_PATH = os.path.join(RESULT_DICT_PATH, 'y_test.csv')
GROUP_TRAIN_PATH = os.path.join(RESULT_DICT_PATH, 'group_train.csv')
SVM_MODEL_PATH = os.path.join(RESULT_DICT_PATH, 'optimized_result.pkl')
if not os.path.exists(RESULT_DICT_PATH): os.makedirs(RESULT_DICT_PATH)

```

```

[4]: def show_class_distribution(labels, title="Class Distribution"):
    """Menampilkan distribusi kelas dalam data label."""
    class_counts = {class_name: labels.count(class_name) for class_name in
↳LABELS}

    # Mempersiapkan data untuk plot
    counts = [class_counts[name] for name in LABELS]

    # Menampilkan plot
    plt.figure(figsize=(10, 6))
    bars = plt.bar(LABELS, counts, color='blue')

    # Menambahkan angka di atas setiap batang
    for bar in bars:
        yval = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2, yval + 5, int(yval),
↳ha='center', va='bottom')

    plt.title(title, fontsize=16)
    plt.xlabel("Nama Kelas", fontsize=12)
    plt.ylabel("Jumlah Gambar", fontsize=12)
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()

```

1.1 Memuat Dataset

Terdapat beberapa dataset yang digunakan seperti Deep Feature, rice_leaves_disease dan paddy_doctor_healthy. Berikut merupakan komposisi dataset yang dimuat pada eksperimen ini.

Dataset	total kelas	bacterial_leaf_blight	blight	brown_spot	normal	tungro	Total
Deep	4	1326	960	1200	0	1308	4794
Feature							
rice_leaves_disease	4	180	305	267	157	0	909
Paddy_doctor_healthy		0	0	0	1400	0	1700
Total	5	1506	1265	1467	1557	1308	7103

```
[5]: # Load data
def load_images_from_directory(base_path, limit=1000):
    data = []
    labels = []
    for folder_name in LABELS:
        data_length = len(data)
        folder_path = os.path.join(base_path, folder_name)
        if not os.path.exists(folder_path):
            print(f"Directory not found: {folder_path}")
            continue

        print(len(os.listdir(folder_path)), "images in", folder_name)
        for i, filename in enumerate(os.listdir(folder_path)):
            if i == limit: break
            image_path = os.path.join(folder_path, filename)
            img = cv2.imread(image_path)
            if img is None:
                print(f"Failed to load image: {image_path}")
                continue

            data.append(cv2.resize(img, IMG_SIZE))
            labels.append(folder_name)
        print(f"Loaded {len(data)-data_length} images from {folder_name}")
    print('dataset loaded')
    print('dataset len:', len(data))
    print('labels len:', len(labels))
    return np.array(data), np.array(labels)
```

```
[24]: all_dataset, all_label = load_images_from_directory(MAIN_DATASET[0],
↳ MAIN_DATASET[1])

for dataset in DATASETS:
    print(f"Loading additional dataset from {dataset[0]}")
    loaded_dataset, loaded_labels = load_images_from_directory(dataset[0],
↳ dataset[1])
    all_dataset = np.concatenate((all_dataset, loaded_dataset), axis=0)
    all_label = np.concatenate((all_label, loaded_labels), axis=0)
```

```

print('extra dataset loaded')
print('dataset shape:', all_dataset.shape)
print('labels shape:', all_label.shape)

```

```

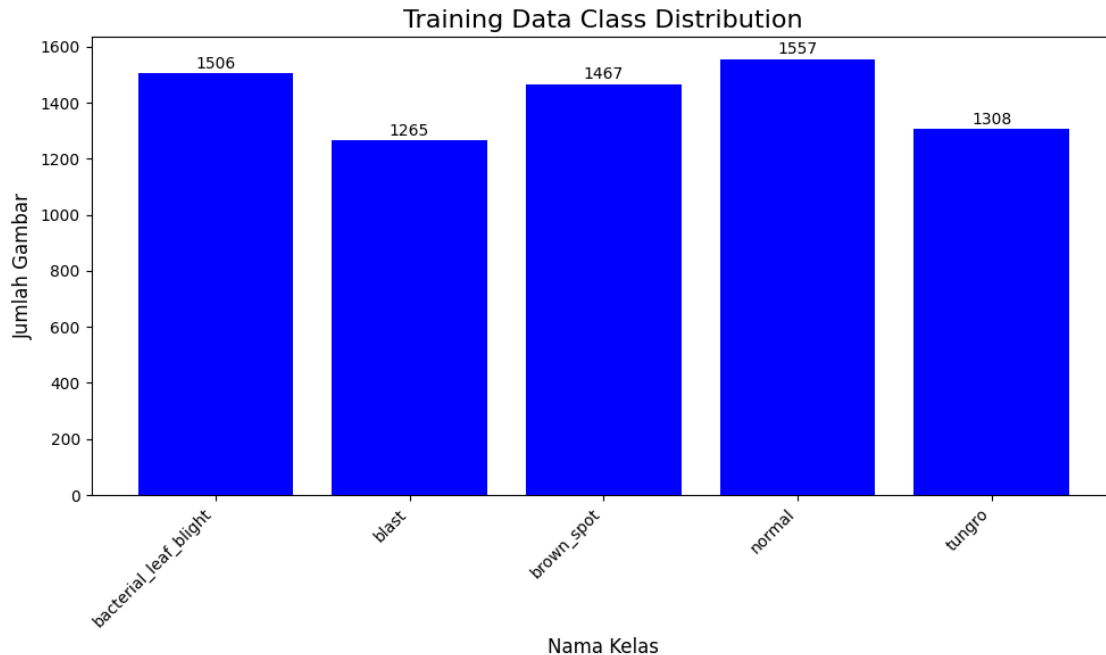
1326 images in bacterial_leaf_blight
Loaded 1326 images from bacterial_leaf_blight
960 images in blast
Loaded 960 images from blast
1200 images in brown_spot
Loaded 1200 images from brown_spot
0 images in normal
Loaded 0 images from normal
1308 images in tungro
Loaded 1308 images from tungro
dataset loaded
dataset len: 4794
labels len: 4794
Loading additional dataset from datasets/paddy-disease-classification-healthy
Directory not found: datasets/paddy-disease-classification-
healthy\bacterial_leaf_blight
Directory not found: datasets/paddy-disease-classification-healthy\blast
Directory not found: datasets/paddy-disease-classification-healthy\brown_spot
1764 images in normal
Loaded 1400 images from normal
Directory not found: datasets/paddy-disease-classification-healthy\tungro
dataset loaded
dataset len: 1400
labels len: 1400
Loading additional dataset from datasets/rice_leaves_disease
180 images in bacterial_leaf_blight
Loaded 180 images from bacterial_leaf_blight
305 images in blast
Loaded 305 images from blast
267 images in brown_spot
Loaded 267 images from brown_spot
157 images in normal
Loaded 157 images from normal
Directory not found: datasets/rice_leaves_disease\tungro
dataset loaded
dataset len: 909
labels len: 909
extra dataset loaded
dataset shape: (7103, 222, 222, 3)
labels shape: (7103,)

```

```

[25]: show_class_distribution(all_label.tolist(), title="Training Data Class_
      ↪Distribution")

```



```
[26]: permutation_indices = np.random.permutation(len(all_dataset))
      all_dataset = all_dataset[permutation_indices]
      all_label = all_label[permutation_indices]
```

Seluruh dataset telah dimuat menjadi satu kumpulan data baik data citra dan juga label/kelas dari masing-masing data tersebut. Dari Diagram tersebut terlihat bahwa terjadi unbalanced/ketidakseimbangan antar kelas baik penyakit maupun yang sehat. Berikutnya adalah proses Preprocessing yang sebelumnya sudah diterapkan yaitu **resize** pada fungsi `load_images_from_directory()` menjadi sesuai dengan ukuran yang terdapat pada variabel `IMG_SIZE` yaitu dengan ukuran 222X222.

1.2 Preprocessing

Pada tahap ini dataset yang sudah dikumpulkan dan dimuat akan di-preprocessing. Terdapat beberapa tahapan yang dilakukan pada preprocessing ini, seperti: 1. pembagian dataset menjadi data training dan validation dengan data testing 2. augmentasi pada data training dan validation 3. normalisasi citra menggunakan CLAHE pada channel L dari color space LAB 4. Konversi dataset yang sudah dinormalisasi ke color space Grayscale, HSV dan LAB

1.2.1 Pembagian Dataset

Pembagian dataset dilakukan untuk membagi dataset menjadi 2 yaitu dataset training dan validasi dengan data testing. Data Testing akan memiliki jumlah yang seimbang yaitu di 200 gambar/kelas sehingga totalnya ada 1000 gambar, sedangkan untuk data training adalah sisanya. Data Testing akan digunakan untuk pengujian terakhir pada model dengan performa dan efisiensi terbaik. Berikut hasil pembagian data training dan validasi dengan data testing.

```
[6]: def split_dataset_by_class(dataset, labels, num_test_per_class):
    """
    Splits the dataset into training and testing sets with a fixed number
    of test samples per class.

    Args:
        dataset (np.ndarray): The feature data of the entire dataset.
        labels (np.ndarray): The labels of the entire dataset.
        num_test_per_class (int): The fixed number of test samples for each
        ↪class.

    Returns:
        tuple: A tuple containing X_train, X_test, y_train, y_test.
    """
    print(f"--- Splitting dataset with a fixed test size of
    ↪{num_test_per_class} per class ---")

    # Get unique classes and their indices
    # unique_classes = np.unique(LABELS)

    # Initialize lists to store the split data
    X_train_list, X_test_list = [], []
    y_train_list, y_test_list = [], []

    # Iterate through each unique class
    for label in LABELS:
        # Find the indices of samples belonging to the current class
        class_indices = np.where(labels == label)[0]
        class_dataset = dataset[class_indices]
        class_labels = labels[class_indices]

        # Check if the class has enough samples for the test set
        if len(class_dataset) < num_test_per_class:
            print(f"Warning: Class {label} has fewer than {num_test_per_class}
            ↪samples. It will be entirely used for testing.")
            # Use all samples for testing if not enough
            X_test_list.append(class_dataset)
            y_test_list.append(class_labels)

        else:
            # Perform a stratified split for the current class
            X_train_class, X_test_class, y_train_class, y_test_class =
            ↪train_test_split(
                class_dataset,
                class_labels,
                test_size=num_test_per_class,
                random_state=42,
```

```

        stratify=class_labels
    )

    # Append the results to the lists
    X_train_list.append(X_train_class)
    X_test_list.append(X_test_class)
    y_train_list.append(y_train_class)
    y_test_list.append(y_test_class)

    # Concatenate the split data from all classes
    X_train = np.concatenate(X_train_list, axis=0)
    X_test = np.concatenate(X_test_list, axis=0)
    y_train = np.concatenate(y_train_list, axis=0)
    y_test = np.concatenate(y_test_list, axis=0)

    return X_train, X_test, y_train, y_test

```

```

[27]: X_train, X_test, y_train, y_test = split_dataset_by_class(all_dataset,
    ↳ all_label, num_test_per_class=200)
    train_group_id = np.arange(len(X_train))
    show_class_distribution(y_train.tolist(), title="Training Data Class
    ↳ Distribution")

```

--- Splitting dataset with a fixed test size of 200 per class ---



Segment	bacterial_leaf_blight	blast	brown_spot	normal	tungro	Total
Train	1306	1065	1267	1357	1108	6403
Test	200	200	200	200	200	1000
total	1506	1265	1467	1557	1308	7103

```
[16]: # Save test data from splitted dataset
def save_data(X, y, output_dir):
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    for i, (image, label) in enumerate(zip(X, y)):
        if not os.path.exists(f"{output_dir}/{label}"):
            os.makedirs(f"{output_dir}/{label}")
        image_path = os.path.join(output_dir, f"{label}/{label}_{i}.jpg")
        cv2.imwrite(image_path, image)

save_data(X_test, y_test, TEST_IMAGES_PATH)
```

```
[17]: fig = plt.figure(figsize=(15, 6))

fig.suptitle('Train Data', fontsize=16)

if len(LABELS) < 5: cols = len(LABELS)
else: cols = 5

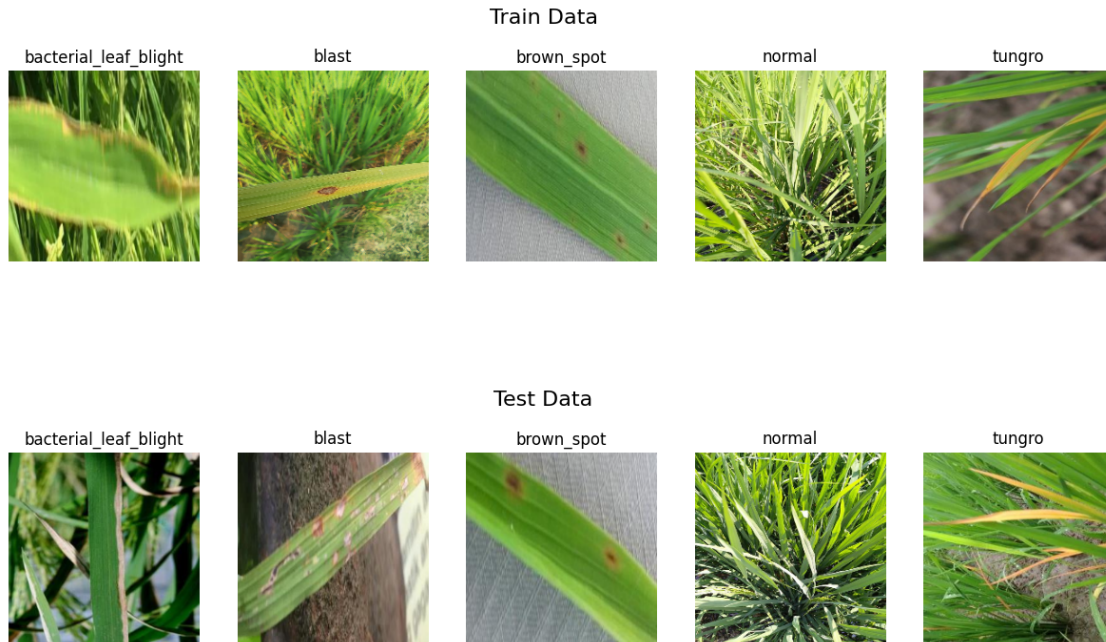
row = (len(LABELS) // 5)+1

for j in range(len(LABELS)):
    fig.add_subplot(row, cols, j+1)
    index = np.where(y_train == LABELS[j])[0][0]
    plt.imshow(cv2.cvtColor(X_train[index], cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title(LABELS[j])

fig = plt.figure(figsize=(15, 6))

fig.suptitle('Test Data', fontsize=16)

for j in range(len(LABELS)):
    fig.add_subplot(row, cols, j+1)
    index = np.where(y_test == LABELS[j])[0][0]
    plt.imshow(cv2.cvtColor(X_test[index], cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title(LABELS[j])
```



1.2.2 Augmentasi data training dan validasi

Augmentasi dilakukan untuk menyeimbangkan jumlah gambar dari masing-masing kelas. Augmentasi yang dilakukan terdiri dari augmentasi warna seperti brightness contrast dan gamma untuk memberikan sedikit variasi dan juga augmentasi geometri seperti flip dan rotation. Target jumlah gambar training dan validasi yang sudah diaugmentasi yaitu 5000 gambar. Agar dapat memenuhi target tersebut, maka setiap gambar akan dipilih secara acak untuk diaugmentasi lebih banyak dibandingkan yang lainnya (oversampling). Selain itu, untuk mencegah terjadinya kebocoran data antara data training dan validasi, maka setiap gambar akan diberi grup antara yang asli dengan hasil augmentasinya, sehingga hasil augmentasi tidak terpisah dengan gambar asli dan masuk di bagian yang berbeda baik training maupun validasi.

```
[7]: def rotate_image(image, angle):
    (h, w) = image.shape[:2]
    center = (w // 2, h // 2)
    M = cv2.getRotationMatrix2D(center, angle, 1.0)
    # Menggunakan BORDER_REPLICATE untuk mengisi piksel kosong dengan piksel
    ↪tepi terdekat
    rotated = cv2.warpAffine(image, M, (w, h), borderMode=cv2.BORDER_REPLICATE)
    return rotated
```

```
[8]: def get_balancer_amount(max_amount):
    return {f'{class_name}': max_amount-y_train.tolist().count(class_name) for
    ↪class_name in LABELS}
```

```

[9]: def augment_images(X, y):
    augment_pool = ['flip_h', 'flip_v', 'rot90', 'rot180', 'rot270']
    balancer_amount = get_balancer_amount(AUGMENTATION_TARGET)

    augmented_group_id = []
    augmented_images = []
    augmented_labels = []

    for class_name, amount in balancer_amount.items():
        write_path = f"{AUGMENTED_IMAGES_PATH}/{class_name}/"
        os.makedirs(f'{write_path}', exist_ok=True)

        indices = np.where(y == class_name)[0]
        original_images = X[indices]

        shuffled_number = np.random.permutation(len(original_images))
        shuffled_images = original_images[shuffled_number]
        shuffled_indices = indices[shuffled_number]

        original_len = y_train.tolist().count(class_name)
        augmentation_len = amount//original_len
        remainder_data = amount%original_len

        for i, img in enumerate(shuffled_images):

            # Pilih fungsi augmentasi secara acak
            if i < remainder_data: chosen_aug = random.sample(augment_pool,
↪augmentation_len+1)
            else: chosen_aug = random.sample(augment_pool, augmentation_len)

            for augmentation in chosen_aug:
                if augmentation == 'flip_h': augmented_image = cv2.flip(img.
↪copy(), 1)
                elif augmentation == 'flip_v': augmented_image = cv2.flip(img.
↪copy(), 0)
                elif augmentation == 'rot90': augmented_image =
↪rotate_image(img.copy(), 90)
                elif augmentation == 'rot180': augmented_image =
↪rotate_image(img.copy(), 180)
                elif augmentation == 'rot270': augmented_image =
↪rotate_image(img.copy(), -90)

            # Tambahkan hasil ke list
            augmented_images.append(augmented_image)
            augmented_group_id.append(shuffled_indices[i])
            augmented_labels.append(class_name)

```

```

        cv2.imwrite(f"{write_path}/
↪{shuffled_indices[i]}_augmented_{augmentation}.jpg", augmented_image)

    return np.array(augmented_images), np.array(augmented_labels), np.
↪array(augmented_group_id)

```

```

[25]: augmented_x_train, augmented_y_train, augmented_group_id =
↪augment_images(X_train, y_train)

```

```

[28]: # Load data with group id
def load_augmented_images_from_directory(base_path, limit=1000):
    data = []
    labels = []
    group_id = []
    for folder_name in LABELS:
        data_length = len(data)
        folder_path = os.path.join(base_path, folder_name)
        if not os.path.exists(folder_path):
            print(f"Directory not found: {folder_path}")
            continue

        print(len(os.listdir(folder_path)), "images in", folder_name)
        for i, filename in enumerate(os.listdir(folder_path)):
            if i == limit: break
            image_path = os.path.join(folder_path, filename)
            img = cv2.imread(image_path)
            if img is None:
                print(f"Failed to load image: {image_path}")
                continue

            data.append(cv2.resize(img, IMG_SIZE))
            labels.append(folder_name)
            group_id.append(int(filename.split("_")[0]))
        print(f"Loaded {len(data)-data_length} images from {folder_name}")
    print('dataset loaded')
    print('dataset len:', len(data))
    print('labels len:', len(labels))
    return np.array(data), np.array(labels), np.array(group_id)

# used to load the augmented images without augmenting from scratch again
augmented_x_train, augmented_y_train, augmented_group_id =
↪load_augmented_images_from_directory(AUGMENTED_IMAGES_PATH,
↪AUGMENTATION_TARGET)

```

```

3694 images in bacterial_leaf_blight
Loaded 3694 images from bacterial_leaf_blight
3935 images in blast
Loaded 3935 images from blast

```

```

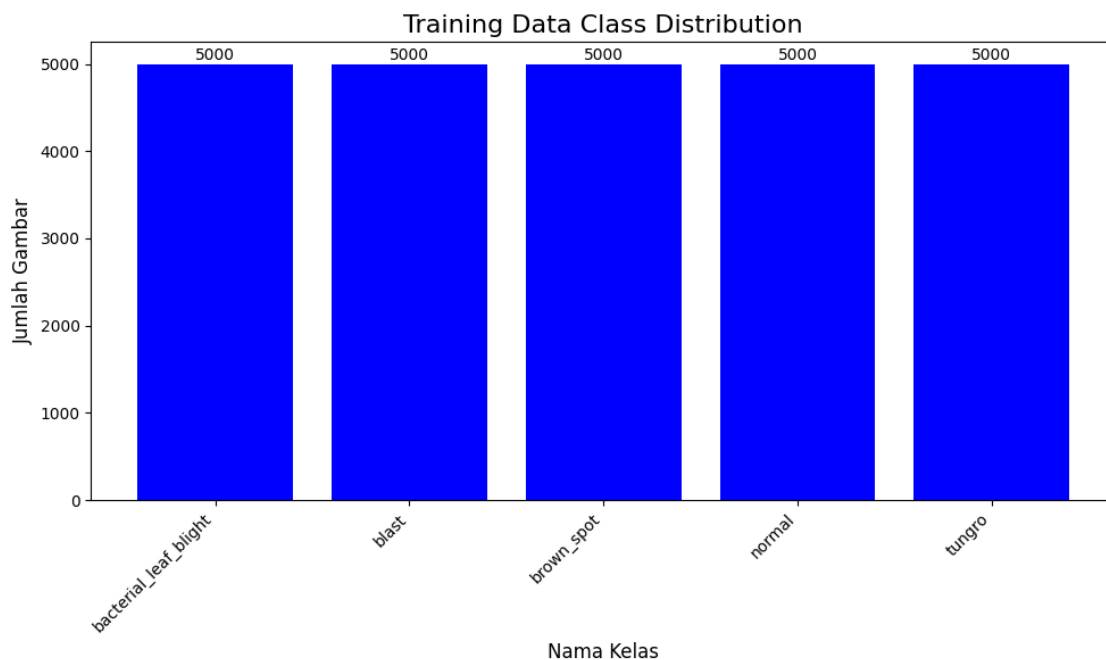
3733 images in brown_spot
Loaded 3733 images from brown_spot
3643 images in normal
Loaded 3643 images from normal
3892 images in tungro
Loaded 3892 images from tungro
dataset loaded
dataset len: 18897
labels len: 18897

```

```

[29]: train_group_id = np.append(train_group_id, augmented_group_id)
X_train = np.append(X_train, augmented_x_train, axis=0)
y_train = np.append(y_train, augmented_y_train)
show_class_distribution(y_train.tolist(), title="Training Data Class_
↪Distribution")

```



```

[ ]: pd.DataFrame(y_train).to_csv(Y_TRAIN_PATH, index=False)
pd.DataFrame(y_test).to_csv(Y_TEST_PATH, index=False)
pd.DataFrame(train_group_id).to_csv(GROUP_TRAIN_PATH, index=False)

```

```

[35]: y_train = pd.read_csv(Y_TRAIN_PATH).to_numpy().ravel()
y_test = pd.read_csv(Y_TEST_PATH).to_numpy().ravel()
train_group_id = pd.read_csv(GROUP_TRAIN_PATH).to_numpy().ravel()

```

```
[41]: first_group_id_indices = np.where(train_group_id == 2314)[0]

if len(first_group_id_indices) < 5:
    cols = len(first_group_id_indices)
    row = 2
else:
    cols = 5
    row = (len(first_group_id_indices) // 5)+1

fig = plt.figure(figsize=(3*cols, 3*row))
fig.suptitle("Group ID 2314", fontsize=16)

for i, j in enumerate(first_group_id_indices):
    fig.add_subplot(row, cols, i+1)
    plt.imshow(cv2.cvtColor(X_train[j], cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title(j)
```



1.2.3 Normalisasi Citra

Keseluruhan data saat ini siap untuk diekstrak fiturnya, namun agar warna citra tersebut dapat lebih stabil dengan tingkat pencahayaan yang berbeda baik saat pagi, siang, maupun sore, maka digunakan normalisasi dengan CLAHE yang dapat melakukan normalisasi secara lokal. Metode ini diterapkan pada channel L pada color space LAB citra tersebut.

```
[63]: def normalizer(image, clahe):
    lab_image = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
    l_channel, a_channel, b_channel = cv2.split(lab_image)

    normalized_l = clahe.apply(l_channel)
    return cv2.cvtColor(cv2.merge((normalized_l, a_channel, b_channel)), cv2.
↪COLOR_LAB2RGB)
```

```
[64]: def dataset_normalizer(dataset):
    normalized_images = []

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

```

for image in dataset: normalized_images.append(normalizer(image, clahe))

return np.array(normalized_images)

```

```

[ ]: normalized_X_train = dataset_normalizer(X_train)
normalized_X_test = dataset_normalizer(X_test)

```

```

[48]: first_group_id_indices = np.where(train_group_id == 2314)[0]

if len(first_group_id_indices) < 5:
    cols = len(first_group_id_indices)
    row = 2
else:
    cols = 5
    row = (len(first_group_id_indices) // 5)+1

fig = plt.figure(figsize=(3*cols, 3*row))
fig.suptitle("Group ID 2314", fontsize=16)

for i, j in enumerate(first_group_id_indices):
    fig.add_subplot(row, cols, i+1)
    plt.imshow(normalized_X_train[j])
    plt.axis('off')
    plt.title(j)

```



1.3 Ekstraksi Fitur

Ekstraksi fitur memungkinkan citra yang masih mentah menjadi kumpulan vektor yang lebih kecil namun bisa memberikan representasi citra yang akurat. Fitur yang akan digunakan terdiri dari fitur warna dan tekstur. Fitur warna yang digunakan yaitu momen warna dari 2 color space yaitu HSV dan LAB, sedangkan fitur tekstur menggunakan LBP dan GLCM.

```

[69]: def calculate_color_moments(image):
    moments = []

    ch1, ch2, ch3 = cv2.split(image)
    for channel in [ch1, ch2, ch3]:

```

```

moments.append(channel[np.nonzero(channel)].mean())
moments.append(channel[np.nonzero(channel)].std())
moments.append(skew(channel.reshape(-1)))
moments.append(kurtosis(channel.reshape(-1)))

```

```

return np.array(moments)

```

```

[70]: def calculate_glm(gray):
    glm = feature.graycomatrix(gray, distances=[8], angles=[0],
    ↪symmetric=True, normed=True)
    contrast = feature.graycoprops(glm, 'contrast')
    energy = feature.graycoprops(glm, 'energy')
    homogeneity = feature.graycoprops(glm, 'homogeneity')
    correlation = feature.graycoprops(glm, 'correlation')
    dissimilarity = feature.graycoprops(glm, 'dissimilarity')

    return np.concatenate((contrast.flatten(), energy.flatten(), homogeneity.
    ↪flatten(), correlation.flatten(), dissimilarity.flatten()))

def calculate_lbp(gray):
    P = 16
    lbp = feature.local_binary_pattern(gray, P, R=2, method='uniform')
    uniform_n_bins = P*(P-1)+3
    hist_lbp, _ = np.histogram(lbp, bins=np.arange(0, uniform_n_bins+1),
    ↪range=(0, uniform_n_bins))
    return np.array(hist_lbp)

```

```

[71]: def feature_extraction(dataset, func, csv_path, color_code):
    feature = []
    for img in dataset:
        feature.append(func(cv2.cvtColor(img, color_code)))

    pd_feature = pd.DataFrame(feature)
    pd_feature.to_csv(csv_path, index=False)
    return pd_feature.to_numpy()

```

```

[76]: momen_X_train_HSV = feature_extraction(normalized_X_train,
    ↪calculate_color_moments, os.path.join(RESULT_DICT_PATH, 'momen_hsv_train.
    ↪csv'), cv2.COLOR_RGB2HSV)
momen_X_test_HSV = feature_extraction(normalized_X_test,
    ↪calculate_color_moments, os.path.join(RESULT_DICT_PATH, 'momen_hsv_test.
    ↪csv'), cv2.COLOR_RGB2HSV)
momen_X_train_LAB = feature_extraction(normalized_X_train,
    ↪calculate_color_moments, os.path.join(RESULT_DICT_PATH, 'momen_lab_train.
    ↪csv'), cv2.COLOR_RGB2LAB)

```

```

momen_X_test_LAB = feature_extraction(normalized_X_test,
    ↳calculate_color_moments, os.path.join(RESULT_DICT_PATH, 'momen_lab_test.
    ↳csv'), cv2.COLOR_RGB2LAB)
X_train_LBP = feature_extraction(normalized_X_train, calculate_lbp, os.path.
    ↳join(RESULT_DICT_PATH, 'lbp_train.csv'), cv2.COLOR_RGB2GRAY)
X_test_LBP = feature_extraction(normalized_X_test, calculate_lbp, os.path.
    ↳join(RESULT_DICT_PATH, 'lbp_test.csv'), cv2.COLOR_RGB2GRAY)
X_train_GLCM = feature_extraction(normalized_X_train, calculate_glcm, os.path.
    ↳join(RESULT_DICT_PATH, 'glcm_train.csv'), cv2.COLOR_RGB2GRAY)
X_test_GLCM = feature_extraction(normalized_X_test, calculate_glcm, os.path.
    ↳join(RESULT_DICT_PATH, 'glcm_test.csv'), cv2.COLOR_RGB2GRAY)

```

```

[28]: momen_X_train_HSV = pd.read_csv(os.path.join(RESULT_DICT_PATH, 'momen_hsv_train.
    ↳csv'))
momen_X_test_HSV = pd.read_csv(os.path.join(RESULT_DICT_PATH, 'momen_hsv_test.
    ↳csv'))
momen_X_train_LAB = pd.read_csv(os.path.join(RESULT_DICT_PATH, 'momen_lab_train.
    ↳csv'))
momen_X_test_LAB = pd.read_csv(os.path.join(RESULT_DICT_PATH, 'momen_lab_test.
    ↳csv'))
X_train_LBP = pd.read_csv(os.path.join(RESULT_DICT_PATH, 'lbp_train.csv'))
X_test_LBP = pd.read_csv(os.path.join(RESULT_DICT_PATH, 'lbp_test.csv'))
X_train_GLCM = pd.read_csv(os.path.join(RESULT_DICT_PATH, 'glcm_train.csv'))
X_test_GLCM = pd.read_csv(os.path.join(RESULT_DICT_PATH, 'glcm_test.csv'))

```

```

[29]: color_feature_X_train = pd.concat([momen_X_train_HSV, momen_X_train_LAB],
    ↳axis=1)
texture_feature_X_train = pd.concat([X_train_LBP, X_train_GLCM], axis=1)
all_feature_X_train = pd.concat([color_feature_X_train,
    ↳texture_feature_X_train], axis=1)

color_feature_X_test = pd.concat([momen_X_test_HSV, momen_X_test_LAB], axis=1)
texture_feature_X_test = pd.concat([X_test_LBP, X_test_GLCM], axis=1)
all_feature_X_test = pd.concat([color_feature_X_test, texture_feature_X_test],
    ↳axis=1)

```

```

[30]: print(momen_X_train_HSV.shape, momen_X_test_HSV.shape, momen_X_train_LAB.shape,
    ↳momen_X_test_LAB.shape, X_train_LBP.shape, X_test_LBP.shape, X_train_GLCM.
    ↳shape, X_test_GLCM.shape)

```

```

(25000, 12) (1000, 12) (25000, 12) (1000, 12) (25000, 243) (1000, 243) (25000,
5) (1000, 5)

```

1.4 Standarisasi, PCA & Pelatihan Model

Standarisasi dilakukan agar model dapat lebih tepat dalam menginterpretasikan fitur tersebut. PCA digunakan sebagai metode yang dapat membantu efisiensi model sehingga model tidak perlu

memproses input yang besar.

```
[31]: pipeline = make_pipeline(StandardScaler(), PCA(), SVC())

param_grid = {
    'pca__n_components': [0.95, 0.90],
    'svc__C': [0.1, 1, 10, 100],
    'svc__gamma': [0.001, 0.01, 0.1, 1],
    'svc__kernel': ['rbf'],
}

sgkf = StratifiedGroupKFold(N_SPLITS, True, 42)
score_list = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
grid_search = GridSearchCV(pipeline, param_grid, cv=sgkf, scoring=score_list,
    ↪refit='f1_macro')
```

```
[32]: def get_fit_result(fit_result):
    pd_fit_result = pd.DataFrame(fit_result.cv_results_)
    return pd_fit_result.sort_values('rank_test_f1_macro').head()[[
        'mean_fit_time',
        'std_fit_time',
        'mean_score_time',
        'std_score_time',
        'param_pca__n_components',
        'param_svc__C',
        'param_svc__gamma',
        'param_svc__kernel',
        'mean_test_accuracy',
        'std_test_accuracy',
        'mean_test_precision_macro',
        'std_test_precision_macro',
        'mean_test_recall_macro',
        'std_test_recall_macro',
        'mean_test_f1_macro',
        'std_test_f1_macro',
        'rank_test_f1_macro',
    ]]

def save_fitted_scenario(fitted_scenario, best_estimator_path, report_path):
    with open(os.path.join(RESULT_DICT_PATH, best_estimator_path), 'wb') as f:
    ↪file:
        pickle.dump(fitted_scenario.best_estimator_, file)

    pd_result = get_fit_result(fitted_scenario)
    pd_result.to_csv(os.path.join(RESULT_DICT_PATH, report_path), index=False)
    return pd_result
```

```
def load_fitted_scenario(best_estimator_path, report_path):
    with open(os.path.join(RESULT_DICT_PATH, best_estimator_path), 'rb') as f:
        model_file:
            model = pickle.load(model_file)

    pd_train_report = pd.read_csv(os.path.join(RESULT_DICT_PATH, report_path))
    return model, pd_train_report
```

```
[36]: result_momen_HSV = copy.deepcopy(grid_search.fit(momen_X_train_HSV, y_train,
    ↪groups=train_group_id))
```

```
[37]: pd_result_momen_HSV = save_fitted_scenario(result_momen_HSV,
    ↪'momen_hsv_scenario.pkl', 'momen_hsv_scenario.csv')
pd_result_momen_HSV
```

```
[37]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
7	5.092002	0.378275	3.557602	0.124324	
27	4.061999	0.282520	2.633803	0.171289	
11	4.646598	0.154465	2.846804	0.227343	
23	4.398196	0.262943	3.452004	0.251329	
15	4.483797	0.127063	2.622408	0.119937	

	param_pca__n_components	param_svc__C	param_svc__gamma	param_svc__kernel	\
7		0.95	1.0		rbf
27		0.90	10.0		rbf
11		0.95	10.0		rbf
23		0.90	1.0		rbf
15		0.95	100.0		rbf

	mean_test_accuracy	std_test_accuracy	mean_test_precision_macro	\
7	0.981679		0.004187	0.981756
27	0.981440		0.003383	0.981547
11	0.981438		0.003764	0.981524
23	0.980077		0.003675	0.980156
15	0.979355		0.004107	0.979442

	std_test_precision_macro	mean_test_recall_macro	std_test_recall_macro	\
7	0.004110		0.981680	0.004145
27	0.003302		0.981456	0.003332
11	0.003698		0.981439	0.003717
23	0.003595		0.980079	0.003620
15	0.004021		0.979349	0.004058

	mean_test_f1_macro	std_test_f1_macro	rank_test_f1_macro
7	0.981681	0.004132	1
27	0.981451	0.003324	2
11	0.981442	0.003706	3

23	0.980074	0.003621	4
15	0.979366	0.004042	5

```
[38]: result_momen_LAB = copy.deepcopy(grid_search.fit(momen_X_train_LAB, y_train,
↳ groups=train_group_id))
```

```
[39]: pd_result_momen_LAB = save_fitted_scenario(result_momen_LAB,
↳ 'momen_lab_scenario.pkl', 'momen_lab_scenario.csv')
pd_result_momen_LAB
```

```
[39]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
7	4.299002	0.113957	3.444805	0.186793	
11	3.982599	0.126240	2.709599	0.121497	
15	3.953803	0.280675	2.616593	0.046182	
14	3.874801	0.262663	2.018602	0.150242	
27	3.302593	0.188046	2.540208	0.258574	

	param_pca__n_components	param_svc__C	param_svc__gamma	param_svc__kernel	\
7		0.95	1.0	1.0	rbf
11		0.95	10.0	1.0	rbf
15		0.95	100.0	1.0	rbf
14		0.95	100.0	0.1	rbf
27		0.90	10.0	1.0	rbf

	mean_test_accuracy	std_test_accuracy	mean_test_precision_macro	\
7	0.982405		0.005406	0.982579
11	0.981889		0.006967	0.982036
15	0.981248		0.006770	0.981418
14	0.976764		0.005852	0.976942
27	0.976081		0.004143	0.976320

	std_test_precision_macro	mean_test_recall_macro	std_test_recall_macro	\
7	0.005205		0.982395	0.005367
11	0.006764		0.981878	0.006930
15	0.006561		0.981241	0.006736
14	0.005661		0.976743	0.005830
27	0.003887		0.976077	0.004101

	mean_test_f1_macro	std_test_f1_macro	rank_test_f1_macro
7	0.982419	0.005342	1
11	0.981902	0.006902	2
15	0.981268	0.006706	3
14	0.976758	0.005807	4
27	0.976104	0.004068	5

```
[40]: result_LBP = copy.deepcopy(grid_search.fit(X_train_LBP, y_train,
↳ groups=train_group_id))
```

```
[41]: pd_result_LBP = save_fitted_scenario(result_LBP, 'lbp_scenario.pkl',
↳ 'lbp_scenario.csv')
pd_result_LBP
```

```
[41]:      mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
11      5.346795      0.541440      5.174405      0.525864
15      7.745798      0.177284      4.319603      0.120005
7       6.939003      0.447538      7.580994      0.386275
31     12.767207      4.270104      5.267794      0.379882
27      8.705391      0.245075      7.895209      0.327926

      param_pca__n_components  param_svc__C  param_svc__gamma  param_svc__kernel  \
11                        0.95          10.0              1.0              rbf
15                        0.95          100.0             1.0              rbf
7                         0.95           1.0              1.0              rbf
31                       0.90          100.0             1.0              rbf
27                       0.90           10.0             1.0              rbf

      mean_test_accuracy  std_test_accuracy  mean_test_precision_macro  \
11          0.920748          0.005513          0.921353
15          0.919788          0.004835          0.920373
7           0.905473          0.005302          0.906022
31          0.870595          0.006990          0.871847
27          0.864710          0.005979          0.866451

      std_test_precision_macro  mean_test_recall_macro  std_test_recall_macro  \
11          0.005263          0.920731          0.005507
15          0.004506          0.919765          0.004829
7           0.005253          0.905451          0.005251
31          0.006948          0.870587          0.006972
27          0.006001          0.864706          0.005841

      mean_test_f1_macro  std_test_f1_macro  rank_test_f1_macro
11          0.920642          0.005528          1
15          0.919701          0.004870          2
7           0.905277          0.005309          3
31          0.870273          0.006973          4
27          0.864289          0.006042          5
```

```
[42]: result_GLCM = copy.deepcopy(grid_search.fit(X_train_GLCM, y_train,
↳ groups=train_group_id))
```

c:\Python312\Lib\site-packages\sklearn\metrics_classification.py:1731:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels
with no predicted samples. Use `zero\_division` parameter to control this
behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Python312\Lib\site-packages\sklearn\metrics_classification.py:1731:

UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

```
[43]: pd_result_GLCM = save_fitted_scenario(result_GLCM, 'glcm_scenario.pkl',
      ↪ 'glcm_scenario.csv')
pd_result_GLCM
```

```
[43]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
15	36.914798	12.668416	12.898597	4.120041	
11	37.840195	4.041062	26.779802	1.789928	
7	36.589801	25.691712	26.204406	8.900203	
14	57.096598	31.894074	22.877009	10.110096	
10	35.540280	27.223342	21.985920	12.188177	

	param_pca__n_components	param_svc__C	param_svc__gamma	param_svc__kernel	\
15		0.95	100.0	1.0	rbf
11		0.95	10.0	1.0	rbf
7		0.95	1.0	1.0	rbf
14		0.95	100.0	0.1	rbf
10		0.95	10.0	0.1	rbf

	mean_test_accuracy	std_test_accuracy	mean_test_precision_macro	\
15	0.753768	0.007700	0.765628	
11	0.716923	0.002843	0.724377	
7	0.681341	0.009057	0.689731	
14	0.672532	0.008809	0.681696	
10	0.660130	0.011803	0.666823	

	std_test_precision_macro	mean_test_recall_macro	std_test_recall_macro	\
15	0.009180	0.753706	0.007943	
11	0.004847	0.716883	0.002933	
7	0.009379	0.681292	0.009155	
14	0.007115	0.672502	0.008750	
10	0.010650	0.660097	0.011739	

	mean_test_f1_macro	std_test_f1_macro	rank_test_f1_macro
15	0.753642	0.008319	1
11	0.716715	0.003481	2
7	0.680382	0.009846	3
14	0.671843	0.008943	4
10	0.658260	0.012342	5

```
[44]: result_color = copy.deepcopy(grid_search.fit(color_feature_X_train, y_train,
      ↪ groups=train_group_id))
```

```
[45]: pd_result_color = save_fitted_scenario(result_color, 'color_scenario.pkl',
↳ 'color_scenario.csv')
pd_result_color
```

```
[45]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
7	9.221205	0.364712	4.201998	0.386182	
11	15.992790	1.124710	6.928848	0.573563	
15	8.964011	0.154862	3.869192	0.232940	
23	15.509413	2.326466	7.473587	1.231487	
27	6.979409	0.212066	3.446591	0.150059	

	param_pca__n_components	param_svc__C	param_svc__gamma	param_svc__kernel	\
7		0.95	1.0	1.0	rbf
11		0.95	10.0	1.0	rbf
15		0.95	100.0	1.0	rbf
23		0.90	1.0	1.0	rbf
27		0.90	10.0	1.0	rbf

	mean_test_accuracy	std_test_accuracy	mean_test_precision_macro	\
7	0.984683	0.004121	0.984866	
11	0.984442	0.003761	0.984631	
15	0.984442	0.003761	0.984631	
23	0.983636	0.004018	0.983709	
27	0.982952	0.004310	0.983020	

	std_test_precision_macro	mean_test_recall_macro	std_test_recall_macro	\
7	0.004024	0.984670	0.004099	
11	0.003669	0.984428	0.003743	
15	0.003669	0.984428	0.003743	
23	0.003946	0.983622	0.004004	
27	0.004259	0.982933	0.004304	

	mean_test_f1_macro	std_test_f1_macro	rank_test_f1_macro
7	0.984691	0.004069	1
11	0.984451	0.003715	2
15	0.984451	0.003715	2
23	0.983635	0.003983	4
27	0.982950	0.004284	5

```
[46]: result_texture = copy.deepcopy(grid_search.fit(texture_feature_X_train,
↳ y_train, groups=train_group_id))
```

```
[47]: pd_result_texture = save_fitted_scenario(result_texture, 'texture_scenario.
↳ pkl', 'texture_scenario.csv')
pd_result_texture
```

```
[47]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
11	7.263804	0.389315	5.516198	0.238363	
15	18.976198	1.579109	11.742802	1.214331	
7	8.335801	0.103040	6.779399	0.184156	
27	14.487008	3.457700	10.161201	1.338481	
31	40.527808	20.358840	14.063391	8.494392	

	param_pca__n_components	param_svc__C	param_svc__gamma	param_svc__kernel	\
11		0.95	10.0	1.0	rbf
15		0.95	100.0	1.0	rbf
7		0.95	1.0	1.0	rbf
27		0.90	10.0	1.0	rbf
31		0.90	100.0	1.0	rbf

	mean_test_accuracy	std_test_accuracy	mean_test_precision_macro	\
11	0.960390	0.003823	0.960658	
15	0.957953	0.003150	0.958224	
7	0.956993	0.003378	0.957354	
27	0.940110	0.004411	0.940667	
31	0.939232	0.003219	0.939801	

	std_test_precision_macro	mean_test_recall_macro	std_test_recall_macro	\
11	0.003681	0.960379	0.003816	
15	0.003042	0.957938	0.003147	
7	0.003209	0.956993	0.003384	
27	0.004228	0.940102	0.004486	
31	0.003030	0.939208	0.003268	

	mean_test_f1_macro	std_test_f1_macro	rank_test_f1_macro
11	0.960370	0.003815	1
15	0.957932	0.003157	2
7	0.956985	0.003381	3
27	0.940073	0.004501	4
31	0.939230	0.003306	5

```
[48]: result_all = copy.deepcopy(grid_search.fit(all_feature_X_train, y_train,
↪groups=train_group_id))
```

```
[49]: pd_result_all = save_fitted_scenario(result_all, 'all_scenario.pkl',
↪'all_scenario.csv')
pd_result_all
```

```
[49]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
14	4.377204	0.282387	2.668400	0.313592	
10	4.184800	0.193136	2.798797	0.128550	
6	4.686602	0.241428	3.297394	0.126308	
26	3.288614	0.320961	2.474988	0.120024	

30	3.583797	0.178845	2.380603	0.089927
----	----------	----------	----------	----------

	param_pca__n_components	param_svc__C	param_svc__gamma	param_svc__kernel \
14	0.95	100.0	0.1	rbf
10	0.95	10.0	0.1	rbf
6	0.95	1.0	0.1	rbf
26	0.90	10.0	0.1	rbf
30	0.90	100.0	0.1	rbf

	mean_test_accuracy	std_test_accuracy	mean_test_precision_macro \
14	0.986762	0.003668	0.986857
10	0.986442	0.003695	0.986550
6	0.984800	0.004213	0.984943
26	0.983036	0.004377	0.983186
30	0.982118	0.004660	0.982275

	std_test_precision_macro	mean_test_recall_macro	std_test_recall_macro \
14	0.003558	0.986767	0.003642
10	0.003549	0.986443	0.003661
6	0.004045	0.984801	0.004164
26	0.004225	0.983040	0.004328
30	0.004473	0.982126	0.004615

	mean_test_f1_macro	std_test_f1_macro	rank_test_f1_macro
14	0.986777	0.003624	1
10	0.986457	0.003646	2
6	0.984817	0.004153	3
26	0.983048	0.004320	4
30	0.982141	0.004595	5

```
[50]: best_model_momen_HSV, pd_result_momen_HSV =
↳ load_fitted_scenario('momen_hsv_scenario.pkl', 'momen_hsv_scenario.csv')
best_model_momen_LAB, pd_result_momen_LAB =
↳ load_fitted_scenario('momen_lab_scenario.pkl', 'momen_lab_scenario.csv')
best_model_LBP, pd_result_LBP = load_fitted_scenario('lbp_scenario.pkl',
↳ 'lbp_scenario.csv')
best_model_GLCM, pd_result_GLCM = load_fitted_scenario('glcm_scenario.pkl',
↳ 'glcm_scenario.csv')
best_model_color, pd_result_color = load_fitted_scenario('color_scenario.pkl',
↳ 'color_scenario.csv')
best_model_texture, pd_result_texture = load_fitted_scenario('texture_scenario.
↳ pkl', 'texture_scenario.csv')
best_model_all, pd_result_all = load_fitted_scenario('all_scenario.pkl',
↳ 'all_scenario.csv')
```

```
[51]: pd_compare_best_result = pd.concat([pd_result_momen_HSV.head(1),  
    ↪pd_result_momen_LAB.head(1), pd_result_LBP.head(1), pd_result_GLCM.head(1),  
    ↪pd_result_color.head(1), pd_result_texture.head(1), pd_result_all.head(1)])  
pd_compare_best_result.sort_values('rank_test_f1_macro')
```

```
[51]:
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
0	5.092002	0.378275	3.557602	0.124324	
0	4.299002	0.113957	3.444805	0.186793	
0	5.346795	0.541440	5.174405	0.525864	
0	36.914798	12.668416	12.898597	4.120041	
0	9.221205	0.364712	4.201998	0.386182	
0	7.263804	0.389315	5.516198	0.238363	
0	4.377204	0.282387	2.668400	0.313592	

	param_pca__n_components	param_svc__C	param_svc__gamma	param_svc__kernel	\
0		0.95	1.0		rbf
0		0.95	1.0		rbf
0		0.95	10.0		rbf
0		0.95	100.0		rbf
0		0.95	1.0		rbf
0		0.95	10.0		rbf
0		0.95	100.0	0.1	rbf

	mean_test_accuracy	std_test_accuracy	mean_test_precision_macro	\
0	0.981679	0.004187	0.981756	
0	0.982405	0.005406	0.982579	
0	0.920748	0.005513	0.921353	
0	0.753768	0.007700	0.765628	
0	0.984683	0.004121	0.984866	
0	0.960390	0.003823	0.960658	
0	0.986762	0.003668	0.986857	

	std_test_precision_macro	mean_test_recall_macro	std_test_recall_macro	\
0	0.004110	0.981680	0.004145	
0	0.005205	0.982395	0.005367	
0	0.005263	0.920731	0.005507	
0	0.009180	0.753706	0.007943	
0	0.004024	0.984670	0.004099	
0	0.003681	0.960379	0.003816	
0	0.003558	0.986767	0.003642	

	mean_test_f1_macro	std_test_f1_macro	rank_test_f1_macro
0	0.981681	0.004132	1
0	0.982419	0.005342	1
0	0.920642	0.005528	1
0	0.753642	0.008319	1
0	0.984691	0.004069	1

0	0.960370	0.003815	1
0	0.986777	0.003624	1

Selain performa dan waktu yang dibutuhkan untuk melatihnya, ukuran model juga digunakan sebagai pertimbangan dalam efisiensinya. Berikut ukuran dari ketujuh model tersebut.

Model	size (KB)
momen_hsv	322
momen_lab	312
lbp	385
glcm	757
color	467
tekstur	442
all	407

1.5 Optimasi Model

Pada tahap ini, model dengan skenario terbaik akan dioptimasi agar hasilnya dapat lebih akurat. Berdasarkan hasil pelatihan pada ketujuh skenario maka didapatkan rentang parameter yang perlu dioptimasi, seperti pca hanya dengan varian sebesar 0.95, nilai C dari 10-100, gamma 0.1 dan 1, dan tetap dengan kernel rbf.

```
[54]: optimized_param_grid = {
      'pca__n_components': [0.95],
      'svc__C': [10, 25, 50, 100],
      'svc__gamma': [0.1, 1],
      'svc__kernel': ['rbf'],
    }

    optimized_grid_search = GridSearchCV(pipeline, optimized_param_grid, cv=sgkf,
    ↪scoring=score_list, refit='f1_macro')
```

```
[56]: optimized_result = optimized_grid_search.fit(all_feature_X_train, y_train,
    ↪groups=train_group_id)
```

```
[ ]: pd_optimized_result = save_fitted_scenario(optimized_result, SVM_MODEL_PATH,
    ↪'optimized_result.csv')
    pd_optimized_result
```

```
[ ]: mean_fit_time std_fit_time mean_score_time std_score_time \
4      4.399199      0.306785      2.551198      0.102433
2      3.997810      0.137940      2.379188      0.185739
6      4.128592      0.236244      2.492402      0.111880
0      4.481841      0.480945      2.616362      0.233501
3     21.200801      1.613669      6.632802      0.590825

    param_pca__n_components param_svc__C param_svc__gamma param_svc__kernel \
```

4	0.95	50	0.1	rbf
2	0.95	25	0.1	rbf
6	0.95	100	0.1	rbf
0	0.95	10	0.1	rbf
3	0.95	25	1.0	rbf

	mean_test_accuracy	std_test_accuracy	mean_test_precision_macro	\
4	0.986842	0.003738	0.986935	
2	0.986802	0.004083	0.986910	
6	0.986762	0.003668	0.986857	
0	0.986442	0.003695	0.986550	
3	0.977115	0.004084	0.978572	

	std_test_precision_macro	mean_test_recall_macro	std_test_recall_macro	\
4	0.003621	0.986845	0.003713	
2	0.003920	0.986807	0.004050	
6	0.003558	0.986767	0.003642	
0	0.003549	0.986443	0.003661	
3	0.003619	0.977102	0.004040	

	mean_test_f1_macro	std_test_f1_macro	rank_test_f1_macro
4	0.986856	0.003694	1
2	0.986818	0.004030	2
6	0.986777	0.003624	3
0	0.986457	0.003646	4
3	0.977312	0.003952	5

```
[ ]: best_model_optimized, pd_optimized_result = load_fitted_scenario(SVM_MODEL_PATH, 'optimized_result.csv')
```

1.6 Pengujian Model

Model yang telah dioptimasi kemudian diuji menggunakan data testing yang sudah di-preprocessing sebelumnya.

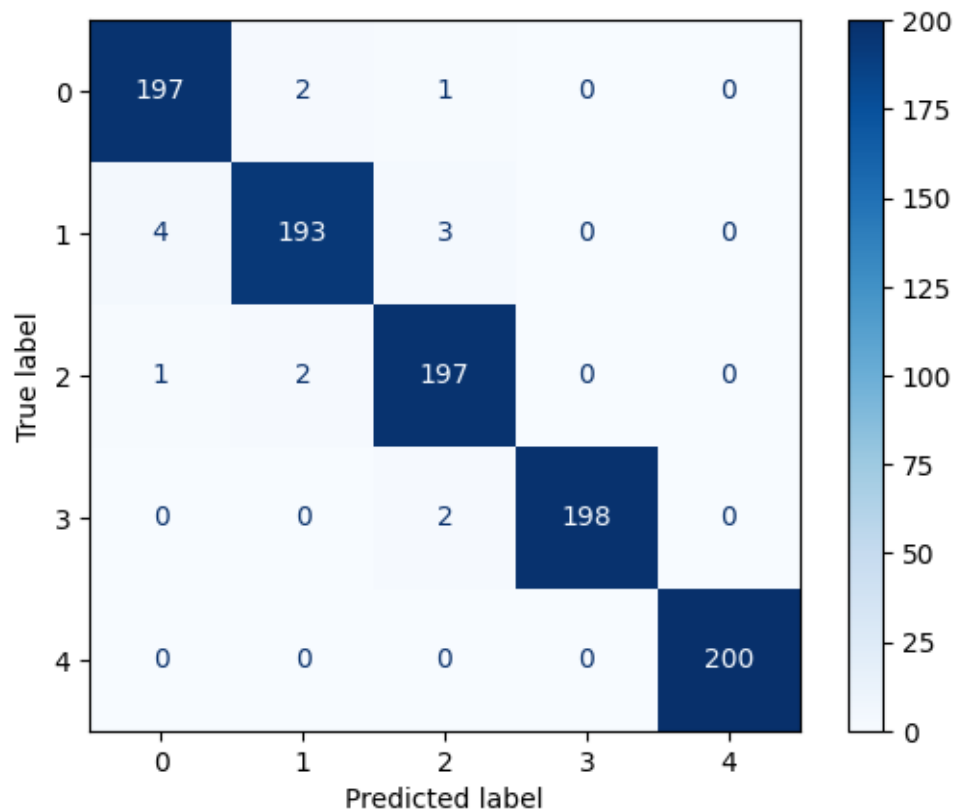
```
[60]: y_pred = best_model_optimized.predict(all_feature_X_test)
# y_pred = svm.predict(x_test_normal)
print(classification_report(y_test, y_pred, digits=4))
cm = confusion_matrix(y_test, y_pred)
print(f"Confusion Matrix for SVM:")
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
print("="*50)
```

	precision	recall	f1-score	support
bacterial_leaf_blight	0.9752	0.9850	0.9801	200
blast	0.9797	0.9650	0.9723	200

brown_spot	0.9704	0.9850	0.9777	200
normal	1.0000	0.9900	0.9950	200
tungro	1.0000	1.0000	1.0000	200
accuracy			0.9850	1000
macro avg	0.9851	0.9850	0.9850	1000
weighted avg	0.9851	0.9850	0.9850	1000

Confusion Matrix for SVM:

=====



1.7 Demo Menggunakan Data baru

Demo ini bertujuan untuk menentukan ketangguhan dari model yang telah dilatih menggunakan dataset yang ada dengan data yang belum sama sekali dilihat bahkan tidak memiliki kemiripan dengan dataset yang ada, hanya ciri-ciri umum seperti background sawah dan lain-lain.

```
[ ]: def extract_features_advanced(image_np_array):

    image = cv2.resize(image_np_array, IMG_SIZE)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

```

        normalized_image = normalizer(image, clahe)
        color_moments_hsv = calculate_color_moments(cv2.cvtColor(normalized_image,
↪cv2.COLOR_RGB2HSV))
        color_moments_lab = calculate_color_moments(cv2.cvtColor(normalized_image,
↪cv2.COLOR_RGB2LAB))
        lbp_feature = calculate_lbp(cv2.cvtColor(normalized_image, cv2.
↪COLOR_RGB2GRAY))
        glcm_feature = calculate_glcm(cv2.cvtColor(normalized_image, cv2.
↪COLOR_RGB2GRAY))

        return np.concatenate((color_moments_hsv, color_moments_lab, lbp_feature,
↪glcm_feature))

```

```

[75]: def test_model_performance(test_data_dir, model_path, classes):
    print("--- Memulai Pengujian Performa Model ---")
    start_total_time = time.time()

    # --- 1. Memuat Model dan Scaler ---
    print(f"\n[{time.strftime('%H:%M:%S', time.localtime())}] Memuat model...")
    try:
        with open(model_path, 'rb') as model_file:
            model = pickle.load(model_file)
    except FileNotFoundError as e:
        print(f"Error: {e}. Pastikan path model sudah benar.")
        return
    print(f"[{time.strftime('%H:%M:%S', time.localtime())}] Model berhasil
↪dimuat.")

    # --- 2. Mengumpulkan Path dan Label Gambar Uji ---
    X_test_paths = []
    y_true_labels = []
    for class_name in classes:
        class_path = os.path.join(test_data_dir, class_name)
        if not os.path.isdir(class_path):
            print(f"Peringatan: Folder '{class_path}' tidak ditemukan. Lewati.")
            continue
        for img_name in os.listdir(class_path):
            if img_name.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp', '.
↪webp')):
                X_test_paths.append(os.path.join(class_path, img_name))
                y_true_labels.append(class_name)

    if not X_test_paths:
        print("Error: Tidak ada gambar yang ditemukan di direktori uji.")
        return

    # --- 3. Melakukan Prediksi dan Mengukur Waktu per Tahap ---

```

```

y_pred_labels = []

total_time_load_img = 0
total_time_extract_feat = 0
total_time_predict = 0

print(f"\n[{time.strftime('%H:%M:%S', time.localtime())}] Memulai loop_
↳prediksi untuk {len(X_test_paths)} gambar...")

for i, img_path in enumerate(X_test_paths):
    # Memuat gambar
    start_stage = time.time()
    img = cv2.imread(img_path)
    end_stage = time.time()
    total_time_load_img += (end_stage - start_stage)

    # Preprocessing dan ekstraksi fitur
    start_stage = time.time()
    features = extract_features_advanced(img)
    end_stage = time.time()
    total_time_extract_feat += (end_stage - start_stage)

    # Prediksi
    start_stage = time.time()
    prediction = model.predict(features.reshape(1,-1))
    y_pred_labels.append(prediction[0])
    end_stage = time.time()
    total_time_predict += (end_stage - start_stage)

    if (i + 1) % 100 == 0 or (i + 1) == len(X_test_paths):
        print(f"[{time.strftime('%H:%M:%S', time.localtime())}] Selesai_
↳memproses {i + 1}/{len(X_test_paths)} gambar.")

end_total_time = time.time()
total_duration = end_total_time - start_total_time

# --- 4. Menampilkan Hasil ---
print("\n--- Hasil Evaluasi ---")
accuracy = accuracy_score(y_true_labels, y_pred_labels)
print(f"Akurasi Prediksi Keseluruhan: {accuracy:.4f}")

print("\n--- Ringkasan Durasi Waktu ---")
print(f"Total Waktu Keseluruhan: {total_duration:.4f} detik")
print(f"Total Gambar Dites: {len(X_test_paths)}")
print(f"\nDurasi Rata-rata per Gambar:")
print(f"  - Memuat Gambar: {total_time_load_img / len(X_test_paths):.6f}_
↳detik")

```

```

    print(f" - Ekstraksi Fitur: {total_time_extract_feat / len(X_test_paths):.
↪6f} detik")
    print(f" - Prediksi Model: {total_time_predict / len(X_test_paths):.6f}↪
↪detik")

```

```
[76]: test_model_performance(BLIND_TEST_IMAGES_PATH, SVM_MODEL_PATH, LABELS)
```

```
--- Memulai Pengujian Performa Model ---
```

```
[07:07:47] Memuat model...
```

```
[07:07:47] Model berhasil dimuat.
```

```
[07:07:47] Memulai loop prediksi untuk 67 gambar...
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
```

```
    warnings.warn(
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
```

```
    warnings.warn(
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
```

```
    warnings.warn(
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
```

```
    warnings.warn(
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
```

```
    warnings.warn(
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
```

```
    warnings.warn(
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
```

```
    warnings.warn(
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
```

```
    warnings.warn(
```

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
```


Durasi Rata-rata per Gambar:

- Memuat Gambar: 0.136150 detik
- Ekstraksi Fitur: 0.089315 detik
- Prediksi Model: 0.002028 detik

```
c:\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature
names
  warnings.warn(
```